Ejercicio 6 - Polimorfismo vía Interfaces

Manuel Rio - 16639

1. Análisis

El backend de un e-commerce se conecta con muchas APIs externas: pasarelas de pago (tarjeta, transferencia, billeteras), empresas de envío y certificadores FEL. No todas hacen lo mismo: unas cobran y reintegran, otras solo cotizan o rastrean, y otras generan documentos.

Requisitos funcionales

- **RF1.** Carga inicial con al menos 10 integraciones de distintos tipos (se cargan 12).
- **RF2.** Listar todas las integraciones con su estado de conexión.
- **RF3.** Buscar una API por su identificador y ver sus parámetros de configuración.
- **RF4.** Ordenar el catálogo por latencia promedio (ms) para ver cuellos de botella.
- **RF5.** Si se pide un reintegro a una API que no maneja dinero (ej. logística), la petición se ignora sin lanzar excepciones ni detener la compra.
- **RF6.** Una integración nueva (la startup de última milla) se agrega solo instanciándola en el catálogo.

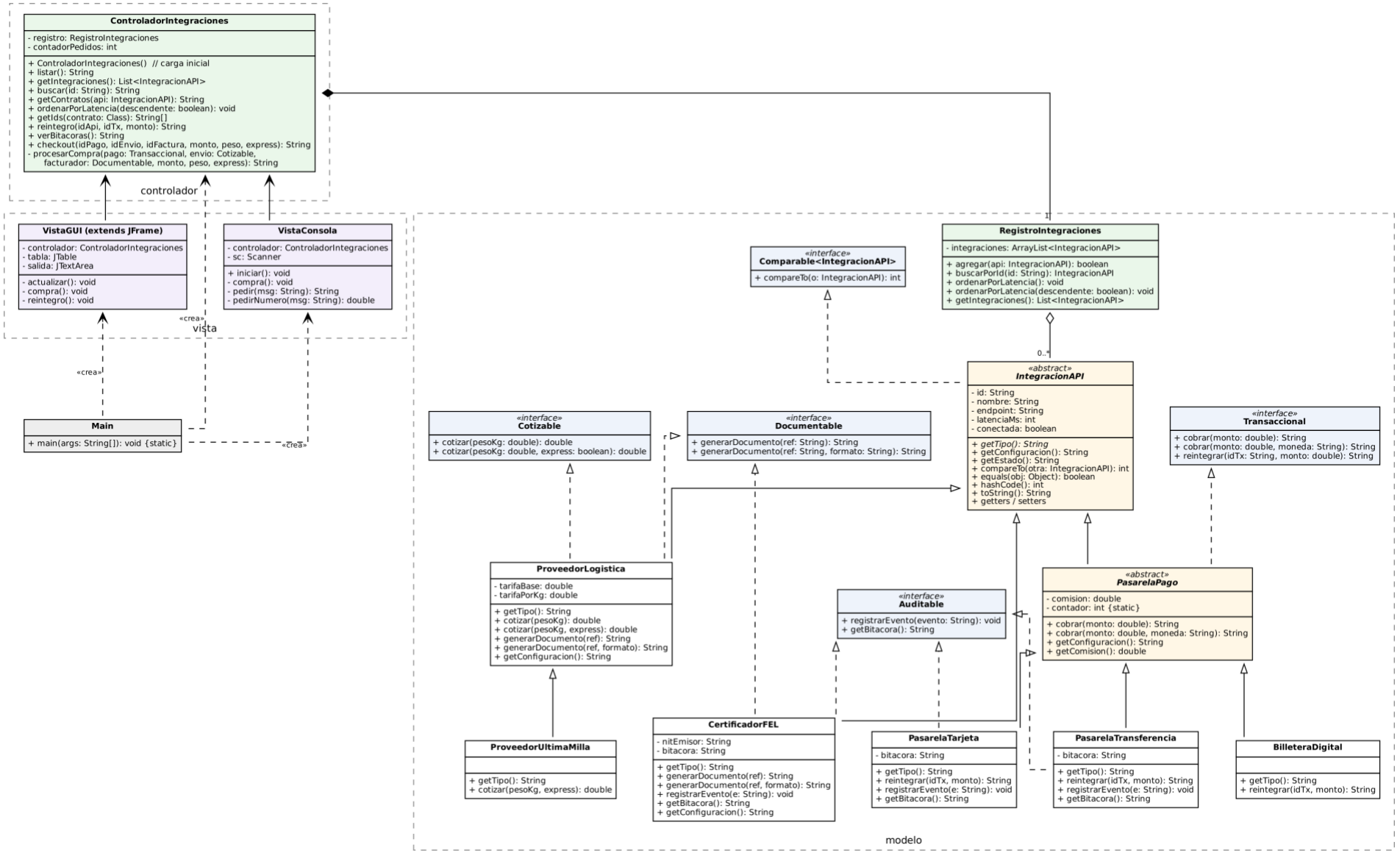
Restricciones de diseño

- Una sola lista polimórfica para guardar todas las integraciones.
- Uso de interfaces (contratos), herencia, overloading y patrón MVC.

Clases identificadas

Clase / interfaz	Tipo	Responsabilidad
IntegracionAPI	Clase abstracta	Datos comunes (id, nombre, endpoint, latencia, estado). Implementa Comparable.
Transaccional	Interfaz	Cobrar y reintegrar dinero.
Documentable	Interfaz	Generar documentos (facturas, guías).
Auditable	Interfaz	Llevar bitácora de eventos.
Cotizable	Interfaz	Dar cotizaciones de envío.
PasarelaPago	Clase abstracta	Base de las pasarelas, implementa Transaccional.
PasarelaTarjeta, PasarelaTransferencia	Clases	Pasarelas que además son Auditable.
BilleteraDigital	Clase	Pasarela que solo es Transaccional (sin auditoría).
ProveedorLogistica	Clase	Envíos: Cotizable y Documentable (guías).
ProveedorUltimaMilla	Clase	Nueva startup, hereda de ProveedorLogistica.
CertificadorFEL	Clase	Facturas FEL: Documentable y Auditable.
RegistroIntegraciones	Modelo	Tiene la única lista polimórfica, búsqueda y ordenamiento.
ControladorIntegraciones	Controlador	Carga inicial, búsqueda, orden, checkout, reintegros y bitácoras.
VistaConsola / VistaGUI	Vistas	Pedir datos y mostrar resultados (consola y Swing).
Main	Driver	Crea el controlador y deja elegir la vista.

2. Diagrama de clases UML



3. Decisiones de diseño

Contratos por tipo de integración

Clase	Transaccional	Documentable	Auditable	Cotizable
PasarelaTarjeta	Sí		Sí	
PasarelaTransferencia	Sí		Sí	
BilleteraDigital	Sí			
ProveedorLogistica		Sí		Sí
ProveedorUltimaMilla		Sí (heredado)		Sí (heredado)
CertificadorFEL		Sí	Sí	

Las capacidades se separaron en interfaces pequeñas en vez de meter todos los métodos en la clase base. Así una API de logística no tiene un método *reintegrar* vacío que no le corresponde, y cada clase implementa solo lo que de verdad hace.

Una sola lista polimórfica

Todas las integraciones se guardan en **ArrayList<IntegracionAPI>** dentro de RegistroIntegraciones. No hay listas separadas por tipo. Cuando se necesita saber qué APIs cumplen un contrato (por ejemplo, para mostrar las pasarelas disponibles) se recorre la misma lista y se pregunta con *instanceof*. La lista se entrega hacia afuera como solo lectura.

Reintegro a una API que no maneja dinero

En *reintegro* el controlador revisa si la integración es *Transaccional*. Si no lo es, devuelve un mensaje diciendo que la petición se ignoró y la compra sigue normal. No se lanza ninguna excepción. Si sí lo es, llama a *reintegrar* a través de la interfaz.

Checkout y extensibilidad

El método privado *procesarCompra(Transaccional pago, Cotizable envio, Documentable facturador, ...)* solo conoce interfaces. Primero cotiza el envío, cobra el total, genera la factura FEL y, si el proveedor de envío también es Documentable, genera la guía. Por eso la startup de última milla (UM-01) funcionó solo agregando una línea en la carga inicial, sin tocar el checkout ni las vistas.

Overloading

- *Transaccional.cobrar(monto)* y *cobrar(monto, moneda)*: la primera usa GTQ por defecto.
- *Documentable.generarDocumento(ref)* y *generarDocumento(ref, formato)*.
- *Cotizable.cotizar(peso)* y *cotizar(peso, express)*.
- *ordenarPorLatencia()* y *ordenarPorLatencia(boolean descendente)* en RegistroIntegraciones.

Buenas prácticas

- **Comparable:** IntegracionAPI implementa *compareTo* comparando la latencia; con eso se usa *Collections.sort* y *Collections.reverseOrder()*.
- **equals / hashCode:** dos integraciones son iguales si tienen el mismo id. El registro lo usa para no aceptar ids repetidos.
- **toString:** en la clase base, da la fila que se muestra al listar. *getConfiguracion()* se sobreescribe en las subclases que tienen datos propios (comisión, tarifas, NIT).
- **Encapsulación:** todos los atributos son privados, con getters y setters donde tiene sentido. El contador de transacciones es static para que los ids no se repitan entre pasarelas.

MVC

El **modelo** son las integraciones, las interfaces y RegistroIntegraciones. El **controlador** recibe las peticiones, decide qué hacer y devuelve el texto a mostrar. Las **vistas** (consola y gráfica) solo piden datos y muestran resultados; las dos usan el mismo controlador, así que la interfaz gráfica no duplicó lógica.

4. Cambios durante la implementación

- **Interfaz Cotizable:** al inicio *cotizarEnvio* estaba directo en ProveedorLogistica. El checkout tenía que hacer cast a esa clase concreta, así que lo pasé a una interfaz.
- **RegistroIntegraciones:** la lista estaba dentro del controlador. La moví a su propia clase del modelo para separar mejor responsabilidades.
- **Scanner compartido:** el Main y la vista de consola abrían dos Scanner sobre System.in; ahora la vista recibe el del Main.

Los tres cambios están comentados en el código y el diagrama ya muestra la versión final.